

ligação interna 430
 listas de argumentos de tamanho variável 426
 long double 432
 long int 432
 long, integer 432
 make 430
 makefile 430
 null 436
 piping 425
 raise 434
 redirecionar a entrada de arquivo 425
 reticências (...) em um protótipo de função 426
 rótulo 437
 signal 434
 <signal.h> 434

símbolo de acréscimo saída >> 426
 símbolo de pipe (|) 425
 símbolo de redirecionamento de entrada < 425
 símbolo de redirecionamento de saída > 425
 static, palavra-chave 429
 <stdlib.h>, arquivo de cabeçalho 436
 tmpfile 433
 unsigned, integer 432
 unsigned long int 432
 va_arg 427
 va_end 427
 va_list 427
 va_start 427
 violações de segmentação 434
 volatile, qualificador de tipo 431

■ Exercício de autorrevisão

14.1 Preencha os espaços em cada uma das sentenças:

- a)** O símbolo de _____ redireciona dados da entrada de um arquivo, e não do teclado.
- b)** O símbolo de _____ é usado para redirecionar a saída da tela, de modo que ela seja gravada em um arquivo.
- c)** O símbolo de _____ é usado para acrescentar a saída de um programa ao final de um arquivo.
- d)** Um(a) _____ direciona a saída de um programa para a entrada de outro programa.
- e)** O uso de _____ na lista de parâmetros de uma função indica que a função pode receber um número variável de argumentos.
- f)** A macro _____ deve ser chamada antes que os argumentos em uma lista de argumentos com tamanhos variáveis possa ser acessada.
- g)** A macro _____ acessa os argumentos individuais de uma lista de argumentos de tamanhos variáveis.
- h)** A macro _____ facilita um retorno normal de uma função cuja lista de argumentos de tamanhos variáveis foi referida pela macro va_start.
- i)** O argumento _____ de main recebe o número de argumentos em uma linha de comando.
- j)** O argumento _____ de main armazena argumentos da linha de comando como strings de caracteres.
- k)** O utilitário _____ do Linux/UNIX lê um arquivo chamado _____, que contém instruções de compilação e ligação de um programa que consiste em vários arquivos-fonte.
- l)** A função _____ força um programa a terminar sua execução.
- m)** A função _____ registra uma função que será chamada no término normal do programa.
- n)** Um _____ de inteiro ou de ponto flutuante pode ser anexado a uma constante inteira ou de ponto flutuante para especificar o tipo exato da constante.
- o)** A função _____ abre um arquivo temporário que existe até que seja fechado ou até que a execução do programa termine.
- p)** A função _____ pode ser usada para interceptar eventos inesperados.
- q)** A função _____ gera um sinal de dentro de um programa.
- r)** A função _____ aloca memória em um array dinamicamente, e inicializa os elementos em zero.
- s)** A função _____ muda o tamanho de um bloco de memória dinâmica previamente alocada.

■ Respostas do exercício de autorrevisão

- 14.1** a) redirecionamento de entrada (<). b) redirecionamento de saída (>). c) acréscimo de saída (>>). d) pipe (|). e) reticências (...). f) va_start. g) va_arg. h) va_end. i) argc. j) argv. k) make, makefile. l) exit. m) atexit. n) sufixo. o) tmpfile. p) signal. q) raise. r) calloc. s) realloc.

Exercícios

- 14.2 *Lista de argumentos de tamanhos variáveis.*** Escreva um programa que calcule o produto de uma série de inteiros que são passados à função `product` por uma lista de argumentos de tamanhos variáveis. Teste sua função com várias chamadas, cada uma com um número diferente de argumentos.
- 14.3 *Impressão de argumentos da linha de comando.*** Escreva um programa que imprima os argumentos da linha de comando do programa.
- 14.4 *Classificação de inteiros.*** Escreva um programa que classifique um array de inteiros em ordem crescente ou decrescente. O programa deve usar argumentos da linha de comando para passar o parâmetro `-c` para a ordem crescente, ou o parâmetro `-d` para a ordem decrescente. [Nota: este é o formato-padrão para passar opções a um programa em UNIX.]
- 14.5 *Arquivos temporários.*** Escreva um programa que coloque um espaço entre cada caractere em um arquivo. Primeiro, o programa deverá gravar o conteúdo do arquivo que está sendo modificado em um arquivo temporário, com espaços entre cada caractere, e depois copiar o arquivo de volta ao arquivo original. Essa operação deverá gravar por cima do conteúdo original do arquivo.
- 14.6 *Tratamento de sinais.*** Leia os manuais de seu compilador para determinar que sinais são aceitos pela biblioteca de tratamento de sinais (`<signal.h>`). Escreva um programa com tratadores de sinal para os sinais `SIGABRT` e `SIGINT`. O programa deve testar a interceptação desses sinais chamando a função `abort` para gerar um sinal do tipo `SIGABRT` e pela digitação de `<Ctrl> c` para gerar um sinal do tipo `SIGINT`.
- 14.7 *Alocação dinâmica de um array.*** Escreva um programa que aloque um array de inteiros dinamicamente. O tamanho do array deve ser fornecido por meio do teclado. Os elementos do array devem ser valores atribuídos a partir da entrada pelo teclado. Imprima os valores do array. Em seguida, realoque a memória do array à metade do número atual de elementos. Imprima os valores restantes do array para confirmar se eles correspondem aos valores da primeira metade do array original.
- 14.8 *Argumentos da linha de comando.*** Escreva um programa que receba dois argumentos da linha de comando que sejam nomes de arquivos, leia os caracteres do primeiro arquivo, um de cada vez, e escreva os caracteres em ordem reversa no segundo arquivo.
- 14.9 *Comando goto.*** Escreva um programa que use comandos `goto` para simular uma estrutura de laços aninhados que imprime um quadrado de asteriscos, como no exemplo a seguir:

```
*****
*      *
*      *
*      *
*      *
*****
```

O programa deverá usar apenas as três instruções `printf` a seguir:

```
printf( "*" );
printf( " ";
printf( "\n" );
```